

MODULE 2 · CORE WORKLOADS

Pods

The smallest deployable unit in Kubernetes

You never run "a container" — you run a Pod

Kubernetes never schedules a bare container. The smallest thing it places on a node is a **Pod** — a wrapper around one or more containers that share a network identity and storage.

Every higher-level object you'll meet — Deployments, StatefulSets, Jobs — exists only to **create and manage Pods**. Understand the Pod and the rest follows.

BY THE END YOU CAN

- Create a Pod two ways
- Read its logs and exec in
- Explain the manifest anatomy
- Say why a bare Pod isn't enough

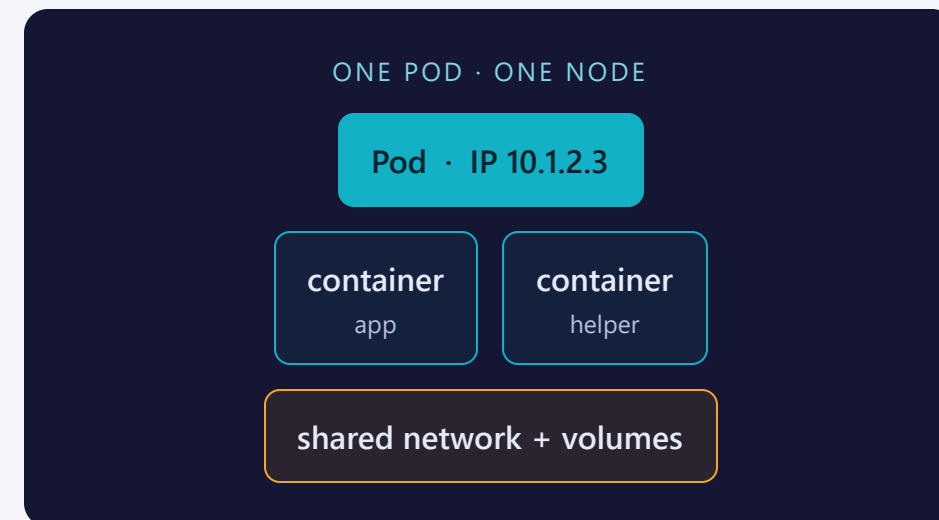
CONCEPT

A Pod is a shared execution context

Containers in one Pod share:

- **Network** — one Pod IP, one localhost, shared ports
- **Storage** — the same mounted volumes
- **Lifecycle** — scheduled together onto one node

Most Pods hold exactly one container. Extra containers are for helpers that must live right next to the app.



Shared, and not shared

SHARED ACROSS CONTAINERS IN THE POD	ISOLATED PER CONTAINER
Network namespace — one IP, shared <code>localhost</code>	Filesystem (own image root)
Mounted volumes	Process table (PIDs)
Node placement & lifecycle	CPU / memory requests & limits

MENTAL MODEL

A Pod is one logical host running cooperating processes — not "a fancy container."

Four parts, every time

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  namespace: training
  labels:
    app: web-pod    # how Services/controllers find it
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
  resources:
    requests:      # reserved by the scheduler
      cpu: "50m"
      memory: "64Mi"
    limits:        # hard ceiling
      cpu: "100m"
      memory: "128Mi"
```

- **apiVersion / kind** — what object this is
- **metadata** — name + labels (the handle everything else uses)
- **spec.containers** — image, ports, command
- **resources** — **requests** the scheduler reserves; **limits** the hard ceiling

The complete manifest — apply with `kubectl apply -f pod.yaml`.

Imperative to explore, declarative to keep

IMPERATIVE — FAST, ONE-OFF

```
kubectl run web --image=nginx:1.25
```

Builds the spec for you, submits it now

DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl apply -f pod.yaml
```

You own the file; it's reviewable and re-appliable

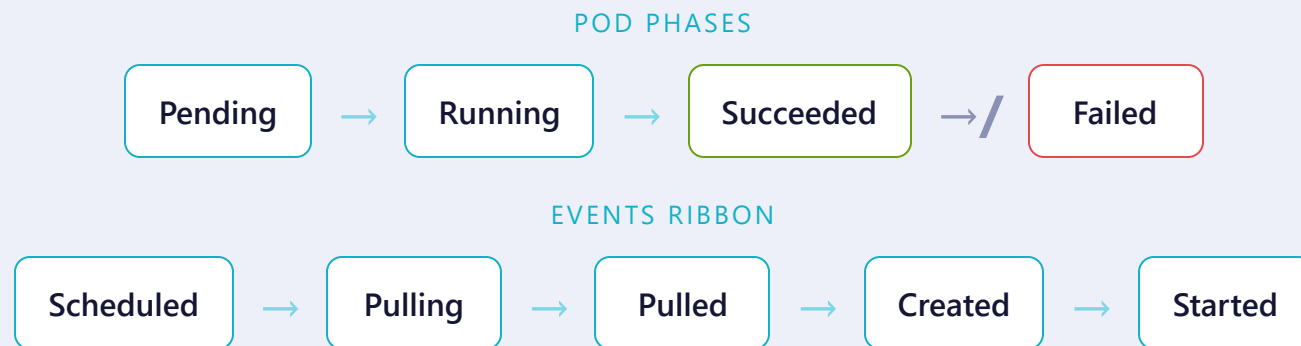
THE BRIDGE TRICK

Generate YAML from an imperative command, then edit & keep it:

```
$ kubectl run web --image=nginx:1.25 \  
  --dry-run=client -o yaml > pod.yaml
```

SEE IT

The lifecycle you'll read in describe



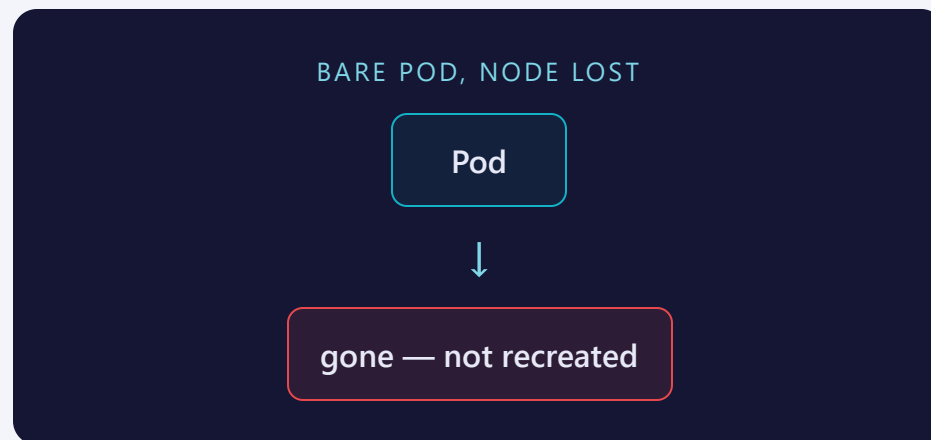
`kubectl describe pod` shows Conditions + this event trail; `kubectl logs` is your first debugging tool.

THE PUNCHLINE

A bare Pod has nobody watching it

Delete a bare Pod and **nothing recreates it**. If its node dies, your app just vanishes — no self-healing, no rescheduling.

That gap is exactly what **Deployments** close (Lab 2.3): a controller that keeps a desired number of Pods alive for you.



Where people trip

- **Treating Pods as durable.** They're cattle, not pets — expect them to be replaced.
- **No requests.** The scheduler can't reserve room; Pods land badly and get evicted first.
- **Reading only get.** Status like Pending hides its reason — describe shows why.
- **Limit too low.** Memory over the limit → OOMKilled; CPU over → throttled, not killed.

Commands to keep at hand

```
$ kubectl run NAME --image=IMG           # create imperatively
$ kubectl get pods -o wide                # node + Pod IP
$ kubectl describe pod NAME               # events, conditions, state
$ kubectl logs NAME                       # container stdout/stderr
$ kubectl exec -it NAME -- sh             # shell inside
$ kubectl get pod NAME -o yaml            # live object + status block
```

Tip: describe before logs — scheduling/pull problems never reach the logs.

RECAP

Pods in one breath

- A Pod = one+ containers sharing network & storage on one node
- Manifest = apiVersion · kind · metadata · spec
- requests reserve, limits cap
- Bare Pods don't self-heal → use controllers

HANDS-ON

Now run Lab 2.1
`labs/2.1/`

Next: [2.2 — Multi-Container Pods](#)